

How Much Can a Difference Be Trusted? Task Validity and Run-to-Run Variation in an Exploratory Evaluation of a Gemma 4 Coding Agent

Six configuration changes, no established improvement, and one tool failure that disappeared without changing which tasks were resolved.

Cristóbal Herrera Jara

Abstract

We tuned the configuration of a Gemma 4 coding agent for the Gemma 4 Developer Agent competition and report, in aggregate counts, what we measured to read our own comparisons. Across six configuration changes we did not establish an improvement; the largest observed increase was not replicated, and this is not evidence of no effect. First, in the competition notebook environment 71 of the 129 public tasks separate the reference patch from no patch; installing three test-only packages raises this to 103. Second, between identical runs 1 to 2 individual tasks changed result while the total changed by 0 to 1 within each pair; one local file submitted twice scored 0.06 and 0.05 on the public leaderboard, and the identity of the two stored files was not verified. Third, the edit tool was called without a mandatory argument in 107 of 134 and 49 of 75 calls in two baseline runs and in 0 of 26 calls in a run with one added instruction line, and the resolved tasks were the same 7 of 30 as in the second baseline run. Fourth, among 23 tasks never resolved in three runs, a retrospective review judged 12 solvable from the issue text and the repository; 10 of those did not reach a relevant edit in at least two of the three runs.

How Much Can a Difference Be Trusted? Task Validity and Run-to-Run Variation in an Exploratory Evaluation of a Gemma 4 Coding Agent

1. Introduction

A participant in this competition submits a configuration for a fixed model and harness and receives the fraction of hidden tasks whose patch passes hidden tests. Development happens on 129 public tasks. With a few dozen tasks per run and a stochastic agent, a change of 1 or 2 resolved tasks is both the effect one hopes for and the effect chance produces.

We ask one question:

In this setting, what difference between two agent configurations can be told apart from repeating the same configuration?

The contribution is what we measured to read our own comparisons:

1. Which public tasks discriminate in the notebook environment, with the cause confirmed by intervention for 31 of the 58 that do not (§ 4.1).
2. The run-to-run variation of a fixed configuration, in task results, totals and failure categories (§ 4.2).
3. Six interventions with predictions logged before each run, including one where a failure mechanism disappeared while the resolved tasks stayed the same (§ 4.3).
4. Where the agent is lost on the tasks it never resolved, with a retrospective judgement of how many were solvable (§ 4.4).

The evidence comes from three sources, kept apart: controls run without the model (§ 4.1), runs of the Gemma 4 agent (§ 4.2 to § 4.4), and a reviewer's judgement (§ 4.4).

2. Related work

SWE-bench poses real GitHub issues whose solution is a patch to the codebase (Jimenez et al., 2024). Later audits of it report solutions given in the issue report or its comments and weak test cases (Aleithan et al., 2024), and patches counted as correct that fail the developer-written test suite (Wang et al., 2026). Zhu et al. (2025) report that many agentic benchmarks have problems in how tasks are set up or outcomes are rewarded. Two public notebooks audited this competition's public tasks before us: Xodarev (2026) reports 119 of 129 sound tasks with a re-implemented grader and rebuilt environments, and Gluzdov (2026) reports 114 after dependency-only repairs, with the official scoring logic unchanged. We did not reproduce them; the three counts differ in environment, control criterion and evaluator.

On SWE-bench Verified, single-run pass rates varied by 2.2 to 6.0 percentage points depending on the run selected, across three models and two scaffolds (Bjarnason et al., 2026). Miller (2024) gives formulas for analysing evaluation data and planning an evaluation. On SWE-bench Verified, performance gains may partly reflect memorisation (Liang et al., 2026). Writing predictions before the data is in the spirit of preregistration (Nosek et al., 2018).

One intervention adds instruction text. Gloaguen et al. (2026) found that repository-level context files did not generally improve task success, although agents followed their instructions.

3. Setting and method

Model and harness. The competition fixes the model, `gemma-4-31b-it-qat-w4a16-ct`, and a harness with tools to run commands, read, search and edit files, and submit a patch. The hidden set has about 120 tasks from private repositories; the 129 public tasks come from four open-source repositories.

Baseline. The official starter kit with no adapters, 8,192 output tokens, model reasoning off, and a per-task budget of 4 minutes, 40 tool calls, 100 turns and 60 seconds per command. Sampling uses temperature 0.2 with no seed.

Runs. One *run* is one pass of one configuration over a fixed task list, in a Kaggle notebook with four L4 GPUs. The measurement set has 30 tasks: 15 from one repository (rich), used throughout development, and 15 from another (fastapi). The fastapi tasks were unseen only in the first of the three 30-task runs. Earlier sessions used those 15 rich tasks, or a 16-task set containing them. All 30 discriminate (§ 4.1). Full traces were stored from the first 30-task run onward.

Loop rule. Before each run we logged a prediction in counts and the outcome that would refute it. Each change in § 4.3 alters one thing. As an operating rule, a change counted as an improvement only with a net gain of at least three resolved tasks; this rule has no statistical test or power analysis behind it.

Deviations, declared. We preregistered a different design: hold out one repository and compare repeated runs with McNemar's test (Dietterich, 1998). We never ran it: the held-out repository was used for development, and variation was measured by counting tasks. All results are therefore exploratory. The prediction log was kept outside version control during this period, and five entries carry estimated times.

4. Results

4.1 The instrument: which public tasks can measure anything

A task *discriminates* if its tests fail without a patch and pass with the reference patch. We ran both controls on all 129 public tasks in the notebook sandbox, without the model (Table 1).

Table 1

Public Tasks That Discriminate, by Repository and Environment

Repository	Tasks	As shipped	With three test packages installed
fastapi	67	29	60
rich	48	42	43
requests and httpx	14	0	0
Total	129	71	103

Note. Each cell counts tasks whose tests fail without a patch and pass with the reference patch. One control run per cell; no model.

As shipped, 58 tasks do not discriminate: in 55 the reference patch does not pass, and 3 pass with no patch. In 34 of the 55 the tests never ran because a package used only by the tests was missing. With the packages installed, 31 of those 34 discriminate. One further rich task changed to discriminating with no identified cause, and none of the original 71 was lost. Twenty-six tasks still do not discriminate: 23 fail with the reference patch, cause unexamined, and 3 pass without a patch.

Of the 58 tasks (45% of 129) that could not measure anything as shipped, 31 (53%) reflect a gap in the environment. Failing to discriminate in this sandbox does not make a task intrinsically invalid.

4.2 Variation of an unchanged configuration

Table 2

Two Runs of the Same Configuration on the Same Tasks

Task set	Resolved, run 1 and run 2	Net change in total	Tasks that change result	Tasks that change failure category
16 rich tasks	4 and 4	0	2	—
15 rich tasks	3 and 3	0	2	6
30 tasks (rich and fastapi)	6 and 7	1	1	13

Note. One pair of runs per row; denominators are the tasks in the set. A dash means the category was not recorded.

Table 2 holds two measures. Task-level, 1 or 2 tasks changed result between identical runs; the total of resolved tasks changed by 0, 0 and 1. Three pairs do not establish a minimum detectable difference for either. Across sessions the spread was wider: seven baseline runs on the 15 shared rich tasks resolved 2, 3 or 4, and up to 3 tasks differed between two of them. The failure category of a task (no patch, patch fails the tests, budget exhausted) changed in 13 of 30

tasks, against 1 of 30 for the result: in these data, a failure table from one run was not a stable guide to what to fix.

On the public leaderboard, the same local archive submitted twice scored 0.06 and 0.05. We could not verify that the two stored files are identical, so this is not a controlled replicate. Read as truncated fractions of a 58-task table, the smallest size consistent with published scores, they are 4 and 3 tasks; that reading is ours.

4.3 Six interventions, no established improvement

Table 3

Each Change Against the Baseline on the Same Tasks

Change	Prediction logged before the run	Resolved: baseline, change	Verdict on the prediction
Reasoning on, 4-minute limit	No gain	4 and 4, 3 of 16	Held, for that budget
100 tool calls instead of 40	More tasks resolved	3 and 3, 2 of 15	Refuted
Public two-stage design (read-only locator, then editor)	More tasks resolved	3 and 3, 3 of 15	Refuted
20 tool calls	At least 2 resolved	2, 2 of 15	Held
Written rule against repeated calls	Does not make the first edit earlier	2, 4 of 15	Held; net gain of two, not replicated
One-line rule for the edit tool	Trapped sessions drop to 1 or fewer	6 and 7, 7 of 30	Held on the mechanism; same tasks resolved

Note. One run per change. Baseline counts are from one or two runs, as shown. "Trapped" means three or more consecutive malformed edit calls.

No row of Table 3 establishes an improvement, which is not evidence of no effect: the largest net increase (2 to 4) was a single run on 15 tasks. With reasoning on and a 4-minute limit, 11 of 13 unresolved sessions ran out of time; that row describes reasoning under that budget and says nothing about a larger one. A seventh condition changing three settings together (reasoning on, 60 calls, 4.5 minutes) has no result at the data cut-off.

The last row is our strongest observation, because it separates a mechanism from an outcome. The agent called the edit tool without its mandatory argument in 107 of 134 calls in one baseline run and 49 of 75 in the other, then repeated the identical failing call: 3 of 30 sessions were trapped in each run. A synthetic call whose string argument is closed with the wrong delimiter reproduces the same malformed arguments in the public parser of the model server; the raw model output was not stored. In the single run with one added instruction line the failure did not occur (0 of 26 calls, 0 trapped sessions). The resolved tasks were the same 7 as in the second baseline run, 6 of them also resolved in the first. The tasks trapped in one baseline run were not resolved in the other, although 2 of the 3 were not trapped there.

Of the 40 predictions logged, 30 were evaluated: 19 held and 11 were refuted. Four could not be evaluated or were never run, and six are pending.

4.4 Where the agent is lost

Twenty-three of the 30 tasks were never resolved in three runs (69 sessions). Fifteen have an issue text shorter than 250 characters: a title and, in most, a reference to an external issue. No short-statement task was resolved: 0 of 15 in each 30-task run, and 0 of 16 counting one more from the earlier 16-task set. The threshold was chosen after seeing the rich data, then held in a prediction logged before the fastapi tasks were first run (0 of 6 short, 3 of 9 long).

A reviewer read the issue, reference patch, hidden tests and surrounding code of all 23 tasks and judged 12 solvable from the issue text and the repository, 9 solvable only by guessing an unstated name, message or value, and 2 not solvable. This judgement is retrospective, made with access to the reference patch and hidden tests; it is not an independent measure of difficulty.

Table 4*Furthest Stage Reached in at Least Two of Three Runs*

Furthest stage	23 never resolved	of which 12 judged solvable	7 resolved
Does not read the file that had to change	10	6	0
Reads it, does not edit it	6	4	0
Edits another region of it	2	0	0
Reaches the right region	5	2	7

Note. Columns count tasks, placed at the highest stage reached in two or more of three runs.

Ten of the 12 tasks judged solvable did not reach a relevant edit in at least two of three runs (Table 4). Of the 69 sessions on never-resolved tasks, 25 never attempted an edit; in the 44 that did, the first attempt came after a median of 65% of the tool-call budget, against 26% in the 20 resolved sessions. Half of their calls (1,267 of 2,532) repeated an identical earlier call, and 36 of the 69 sessions made 40 or more tool calls, while 5 ended with the harness's session-timeout error (3 of them among the 36). Two further tool defects waste calls: the file-reading tool lost its line range in 344 of 851 calls across the three runs, in resolved and failed sessions alike, and the similarity search returned nothing in all 221 calls.

When a patch exists it is rarely close: of the 41 of those sessions with a patch, 16 contain only debugging scripts, the agent's own tests or documentation, and in 23 of the other 25 the agent passes none of the target tests. No verification failure was caused by the environment.

5. Threats to validity

- **Small sample.** Thirty tasks, two repositories, three runs; one run for the edit-tool rule.
- **Exploratory.** The preregistered design was not run, development tasks were reused, and the operating rule has no test or power analysis.
- **Judgement.** Solvability is one retrospective reading. The trace, test and solvability analyses were produced by LLM-based assistants under the author's direction, not

independently replicated.

- **Public is not hidden.** The organisers state that the hidden tasks were curated separately; nothing here predicts the hidden score.

6. Reproducibility and data use

Data cut-off: October 7, 2026, 01:20 UTC. The baseline configuration has fingerprint 3b0e87556166 and the edit-tool variant 7e2082197834: the first 12 hexadecimal digits of the SHA-256 of the base64 encodings of the six configuration files, sorted as strings and concatenated with no names or separators. The validity scripts and leaderboard reader are public: https://github.com/cherrera0001/Agents_Learning_Loops. The competition forbids redistributing task content, so per-task results and traces are not published.

7. Conclusion

Between identical runs, 1 or 2 tasks changed result, the total changed by at most one within a pair and from 2 to 4 of 15 across seven baseline runs, and the failure category changed in 13 of 30 tasks; we did not read differences of that size as effects, and we claim no detection threshold. Fifty-eight of 129 public tasks could not measure anything in the environment as shipped. Across six configuration changes we did not establish an improvement. In one run, a tool failure did not occur and the resolved tasks were the same 7 as in the second baseline run. In 16 of the 23 tasks never resolved, the agent did not edit the file that had to change in at least two of three runs.

References

- Aleithan, R., Xue, H., Mohajer, M. M., Nnorom, E., Uddin, G., & Wang, S. (2024). *SWE-Bench+: Enhanced coding benchmark for LLMs*. arXiv. <https://doi.org/10.48550/arXiv.2410.06992>
- Bjarnason, B. H., Silva, A., & Monperrus, M. (2026). *On randomness in agentic evals*. arXiv. <https://doi.org/10.48550/arXiv.2602.07150>
- Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7), 1895–1923. <https://doi.org/10.1162/089976698300017197>
- Gloaguen, T., Mündler-Sasahara, N., Müller, M. N., Raychev, V., & Vechev, M. (2026). *Evaluating AGENTS.md: Are repository-level context files helpful for coding agents?* arXiv. <https://doi.org/10.48550/arXiv.2602.11988>
- Gluzdov, D. (2026). *Gemma 4: Measure before you tune* (Version 7, last run October 1, 2026) [Computational notebook]. Kaggle. Retrieved October 4, 2026, from <https://www.kaggle.com/code/dmitriigluzdov/gemma-4-measure-before-you-tune>
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VTF8yNQM66>
- Liang, S., Garg, S., & Moghaddam, R. Z. (2026). The SWE-Bench illusion: When state-of-the-art LLMs remember instead of reason. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice* (pp. 395–405). Association for Computing Machinery. <https://doi.org/10.1145/3786583.3786882>
- Miller, E. (2024). *Adding error bars to evals: A statistical approach to language model evaluations*. arXiv. <https://doi.org/10.48550/arXiv.2411.00640>

- Nosek, B. A., Ebersole, C. R., DeHaven, A. C., & Mellor, D. T. (2018). The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11), 2600–2606. <https://doi.org/10.1073/pnas.1708274114>
- Wang, Y., Pradel, M., & Liu, Z. (2026). Are "solved issues" in SWE-bench really solved correctly? An empirical study. In *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering* (pp. 169–181). Association for Computing Machinery. <https://doi.org/10.1145/3744916.3764576>
- Xodarev, A. (2026). *119 of 129 sound: The Gemma 4 grader, rebuilt* (Version 22, last run October 1, 2026) [Computational notebook]. Kaggle. Retrieved October 4, 2026, from <https://www.kaggle.com/code/busyaprime/119-of-129-sound-the-gemma-4-grader-rebuilt>
- Zhu, Y., Jin, T., Pruksachatkun, Y., Zhang, A., Liu, S., Cui, S., Kapoor, S., Longpre, S., Meng, K., Weiss, R., Barez, F., Gupta, R., Dhamala, J., Merizian, J., Giulianelli, M., Coppock, H., Ududec, C., Kellermann, A., Sekhon, J., . . . Kang, D. (2025). Establishing best practices in building rigorous agentic benchmarks. In *Advances in Neural Information Processing Systems 38* (pp. 184435–184475). Neural Information Processing Systems Foundation. <https://doi.org/10.52202/085713-5547>